

09. gyakorlat - Prímtesztelés, while ciklus és szimuláció

A gyakorlat először három prímtesztelő megoldást hasonlít össze, majd while ciklussal ismételt véletlen kísérleteket. A nagyon nagy teszteseteknél előbb becsüld meg a ciklus hosszát: egyes sorok szándékosan nagyon lassúak lehetnek.

A tanári változat nemcsak a várható eredményt írja le, hanem azt is, milyen R-szabályt érdemes a kódból felismerni. A véletlen mintáknál konkrét szám helyett a működést és a várható mintázatot emeli ki.

Prímtesztelés - hatékonyság

A három változat ugyanazt az ötletet finomítja: minél kevesebb lehetséges osztót vizsgálunk, és amint biztos a válasz, érdemes kilépni.

```
prime1=function(n){  
  ind=T  
  for(i in 2:(n-1)){  
    if(n%i==0){  
      ind=F  
    }  
  }  
  if(ind==T | n==2){  
    print("Ez a szám prímszám")  
  }  
  else if(ind==F){  
    print("Ez a szám nem prímszám")  
  }  
}  
prime1(2)  
prime1(24353)  
prime1(213234233) #a for ciklus 213.234.233 oszthatóságot vizsgál
```

Magyarázat / tanulság

A prime1 majdnem minden 2 és n-1 közötti osztót végigpróbál, ezért nagy n-nél rendkívül lassú. 24353 összetett (7-tel osztható), de a függvény ennek megtalálása után sem áll meg. A prime1(213234233) nagyságrendileg több százmillió cikluslépés, ezért tantermi gépen nem célszerű kivárni. n=2-t külön OR feltétel kezel.

```
prime2=function(n){
  ind=T
  for(i in 2:floor(sqrt(n))){
    if(n%i==0){
      ind=F
    }
  }
  if(ind==T | n==2){
    print("Ez a szám prímszám")
  }
  else if(ind==F){
    print("Ez a szám nem prímszám")
  }
}
prime2(2)
prime2(24353)
prime2(213234233) #a for ciklus 14.062 oszthatóságot vizsgál meg
floor(sqrt(213234233))
```

Magyarázat / tanulság

A prime2 már csak a négyzetgyökig vizsgál osztókat, ezért sokkal gyorsabb.

$\text{floor}(\text{sqrt}(213234233)) = 14602$. A forrás megjegyzésében szereplő "14.062" szám elírás; a helyes felső határ 14 602. Ettől még a függvény osztó megtalálása után sem lép ki korábban.

```
prime2(9834652638745876)
```

Magyarázat / tanulság

Ez a szám páros, tehát az első vizsgált osztó rögtön bizonyítaná, hogy nem prím. A prime2 azonban nem return-öl azonnal, ezért a négyzetgyökig - közel 99 millió lépésig - folytathatja a ciklust. Ez szándékosan jól mutatja a hatékonysági problémát. Nagyon nagy egész számoknál a szokásos R numeric típus pontossági korlátaira is figyelni kell.

```
prime3=function(n){
  for(i in 2:floor(sqrt(n))){
    if(n%i==0){
      return("Ez a szám nem prímszám")
    }
  }
  return("Ez a szám prímszám")
}
prime3(24353)
prime3(213234233)
prime3(983465263874587687698764329438)
```

Magyarázat / tanulság

A prime3 az első megtalált osztónál azonnal return()-nel kilép. 24353 esetén már 7-nél, 213234233 esetén 431-nél megáll. A legnagyobb tesztszám páros, ezért már $i=2$ -nél visszatér. Megjegyzés: a forráskód $n=2$ esetét nem kezeli helyesen a $2:\text{floor}(\text{sqrt}(n))$ tartomány miatt.

While ciklus

A while addig ismétli a blokkot, amíg a feltétel TRUE. Emiatt különösen fontos, hogy a cikluson belül valami előbb-utóbb hamissá tegye a feltételt.

```
while(feltétel){  
  SZÁMOLÁS  
}
```

```
n=1  
if(n==1){  
  print("alma")  
}  
while(n==1){  
  print("alma")  
  n=2  
}
```

Magyarázat / tanulság

Az if egyszer ellenőrzi a feltételt és egyszer kiírja az "alma" szót. A while belép, mert n=1, egyszer szintén kiírja, majd n=2-re változik, ezért a következő ellenőrzésnél megáll. A teljes rész így kétszer írja ki az "alma" szót.

Dobókocka: első hatos

A cél annak vizsgálata, hány dobás kell az első 6-osig. Először egy rögzített hosszúságú mintát nézünk, majd valódi while ciklust készítünk.

```
dobas1=sample(1:6,6,replace=T)  
dobas1  
for(i in 1:length(dobas1)){  
  if(dobas1[i]==6){  
    print(i)  
  }  
}  
dobas1
```

Magyarázat / tanulság

Hat dobást generál előre, majd kiírja minden 6-os pozícióját. Ez még nem pontosan "dobunk az első hatosig": ha nincs 6-os, semmit nem ír ki; ha több van, több pozíciót is kiír. Jó előkészítés a következő while megoldáshoz.

```
kockadobas=function(){
  ind=T
  dobasokszama=0
  while(ind){
    dobas=sample(1:6,1)
    dobasokszama=dobasokszama+1
    if(dobas==6){
      ind=F
      return(dobasokszama)
    }
  }
}
```

Magyarázat / tanulság

Minden körben egy új dobást végez és növeli a számlálót. Amint 6-ost kap, return()-nel visszaadja a dobások számát. A visszatérési érték véletlen, legalább 1.

```
mean(replicate(100000,kockadobas()))
```

Magyarázat / tanulság

100 000-szer lefuttatja a kísérletet, majd átlagolja a dobásszámot. Egy szabályos kockánál az első 6-osig szükséges dobások elméleti várható értéke 6, ezért az eredmény nagy mintában 6 körül lesz.

Pénzérme: F-I-F minta

A dobásokat addig bővítjük, amíg az utolsó három eredmény pontosan F-I-F nem lesz.

Példa a forrásból: F,I,I,F,F,I,I,F,I,F -> a F-I-F minta a 10. dobásnál fejeződik be.

```
penzermel=function(){
  dobasok=sample(c("F","I"),2,replace=T)
  ind=T
  while(ind){
    kovetkezo_dobas=sample(c("F","I"),1)
    dobasok=c(dobasok,kovetkezo_dobas)
    n=length(dobasok)
    if(dobasok[n-2]=="F" & dobasok[n-1]=="I" & dobasok[n]=="F"){
      ind=F
      return(n)
    }
  }
}
```

Magyarázat / tanulság

Két kezdő dobás után egyesével bővíti a vektort. Mindig csak az utolsó három dobást vizsgálja. Amikor azok F-I-F mintát adnak, visszatér az addigi dobások számával.

```
mean(replicate(10000,penzerme1()))
```

Magyarázat / tanulság

10 000 kísérlet átlagát számolja. Szabályos pénzérménél az F-I-F minta várakozási idejének elméleti várható értéke 10 dobás, ezért a szimulációs átlag 10 körül várható.

Verseny két pénzérmeminta között

A penzerme2 azt vizsgálja, melyik minta jelenik meg előbb: F-I-I vagy F-F-I.

```
penzerme2=function(){  
  dobasok=sample(c("F","I"),2,replace=T)  
  ind=T  
  while(ind){  
    kovetkezo_dobas=sample(c("F","I"),1)  
    dobasok=c(dobasok,kovetkezo_dobas)  
    n=length(dobasok)  
    if(dobasok[n-2]=="F" & dobasok[n-1]=="I" & dobasok[n]=="I"){  
      ind=F  
      return(0)  
    }  
    else if(dobasok[n-2]=="F" & dobasok[n-1]=="F" & dobasok[n]=="I"){  
      ind=F  
      return(1)  
    }  
  }  
}  
penzerme2()
```

Magyarázat / tanulság

Ha előbb F-I-I jelenik meg, 0-t ad vissza; ha előbb F-F-I, akkor 1-et. Az else if biztosítja, hogy egy körben csak az egyik kimenet történjen.

```
s=replicate(100000,penzerme2())  
length(s[s==0])  
length(s[s==1])
```

Magyarázat / tanulság

100 000 versenyt futtat, majd megszámolja a két kimenetet. Elméletileg az F-I-I minta nyerési valószínűsége 1/3, az F-F-I-é 2/3, ezért nagy mintában nagyjából 33 000 darab 0 és 67 000 darab 1 várható.